1 As part of a security system, the user is asked to choose a 4-digit value for their PIN. This PIN cannot be 1234 or 4321

Create an algorithm that:

- takes the PIN as input as a string
- checks that the PIN is 4 characters in length
- checks that the PIN is **not** either “1234” or “4321”
- outputs “`VALID PIN`” if the PIN meets all of the criteria
- outputs “`INVALID PIN`” if the PIN does not meet all of the criteria.

You do **not** need to check that the digits entered are numeric.

[6]

2 OCR Rockfest is a yearly music festival. Visitors buy tickets to attend and watch the music acts.

A computer system is created to sell tickets and store information about visitors and the music acts.

There are 500 tickets available for the music festival. The number of tickets currently available is stored in the variable `tickets`

Complete the algorithm to:

- prompt the user to enter how many tickets they would like and take this value as input
- output "`Tickets booked`" and reduce the value stored in `tickets` if there are enough tickets available
- output "`Not enough tickets`" if there are not enough tickets available
- repeat the steps until there are no tickets left.

You must use either:
- OCR Exam Reference Language, or
- a high-level programming language that you have studied.

```
tickets = 500
```

[6]

3(a) Students take part in a sports day. The students are put into teams.

Students gain points depending on their result and the year group they are in. The points are added to the team score.

The team with the most points at the end of the sports day wins.

Data about the teams and students is stored in a sports day program.

i. Identify the most appropriate data type for each variable used by the program.

Each data type must be different.

Variable	Example	Data type
`teamName`	`"Super-Team"`	
`studentYearGroup`	`11`	
`javelinThrow`	`18.2`	

[3]

© OCR 2026. You may photocopy this page.

Created in ExamBuilder

ii. The student names for a team are stored in an array with the identifier `theTeam`

An example of the data in this array is shown:

Index	0	1	2	3	4	5
Data	Ali	Eve	Ling	Nina	Sarah	Tom

theTeam

A linear search function is used to find whether a student is in the team. The function:

- takes a student name as a parameter
- returns `True` if the student name is in the array
- returns `False` if the student name is **not** in the array.

Complete the design of an algorithm for the linear search function.

```
function linearSearch(studentName)

   for count = 0 to ……..……..……

     if theTeam[……..……..……..……] == ……..……..……..…… then

       return ……..……..……

     endif

   next count

   return False

endfunction
```

[4]

(b) This algorithm calculates the number of points a student gets for the distance they throw in the javelin:

```
01 javelinThrow = input("Enter distance")

02 yearGroup = input("Enter year group")

03 if javelinThrow >= 20.0 then

04 score = 3

05 elseif javelinThrow >= 10.0 then

06 score = 2
```

```
07 else

08 score = 1

09 endif

10 if yearGroup != 11 then

11 score = score * 2

12 endif

13 print("The score is", score)
```

Complete the trace table for the algorithm when a student in year 10 throws a distance of 14.3

You may not need to use all the rows in the table.

Line number	javelinThrow	yearGroup	score	Output

[4]

© OCR 2026. You may photocopy this page.

Created in ExamBuilder

(c) The height a student jumps in the high jump needs to be input and validated.
The height is entered in centimetres (cm) and must be between 40.0 and 180.0 inclusive.

i. Write an algorithm to:

Each data type must be different.

- take the height jumped as input
- output "`VALID`" **or** "`NOT VALID`" depending on the height input.

You must use **either**:
- OCR Exam Reference Language, **or**
- A high-level programming language that you have studied.

---

[4]

ii. The algorithm is tested using a range of tests.

Complete the table to identify an example of test data for each type of test.

Test data (height jumped in cm)	Type of test	Expected output
	Normal	"VALID"
	Boundary	"VALID"
	Erroneous	"NOT VALID"

[3]

(d) The individual results for each student in each event are stored in a database.

The database table `TblResult` stores the times of students in the 100 m race. Some of the data is shown:

StudentID	YearGroup	TeamName	Time
11GC1	11	Valiants	20.3
10VE1	10	Super-Team	19.7
10SM1	10	Super-Team	19.2
11JP2	11	Champions	19.65

Complete the SQL statement to show the Student ID and team name of all students who are in year group 11

```
SELECT StudentID, ...............................................

FROM ...............................................................

...................................................................
```

[4]

(e) Abstraction and decomposition have been used in the design of the sports day program.

i. Identify **one** way that abstraction has been used in the design of this program.

---

[1]

ii. Identify **one** way that decomposition has been used in the design of this program.

---

[1]

(f) An algorithm works out which team has won (has the highest score).

Write an algorithm to:

- prompt the user to enter a team name and score, or to enter "stop" to stop entering new teams
- repeatedly take team names and scores as input until the user enters "stop"
- calculate which team has the highest score
- output the team name and score of the winning team in an appropriate message.

You must use **either**:

- OCR Exam Reference Language, **or**
- A high-level programming language that you have studied

[6]

4 Write an algorithm in pseudocode to:

- store "`Hello`" in the variable `word1`
- store "`Everyone`" in the variable `word2`
- concatenate `word1` and `word2` to store "`HelloEeveryone`" in the variable `message`

---

[3]

END OF QUESTION PAPER

Question			Answer/Indicative content	Marks	Guidance

1			• Input **and** storage/use of PIN
			• Attempt at selection / condition controlled loop…
			• …Correctly checks for **4** digits
			• …Correctly checks for **both** 1234, 4321
			• Correct output of “INVALID PIN” after sensible attempt at <u>all three</u> checks
			• Correct output of “VALID PIN” after sensible attempt at <u>all three</u> checks, **only** when PIN is fully valid and no contradictory output
			(e.g. printing “invalid PIN” as well)
			
			If diagram/flowchart given, ignore shapes if incorrect and mark text inside as algorithm.
				6	<u>Example code</u>
					`pin = input(“Enter PIN”)`
					`if pin.length == 4 and pin != “1234” and pin != “4321” then`
					`  print(“VALID PIN”)`
					`else:`
					`  print(“INVALID PIN”)`
					
					Note - | or || means OR in some languages. AND is &.
					
					BOD input and use/comparison of PIN as integer. No need for quotation marks around 1234 / 4321. Allow equivalent outputs.
					
					BP3 and 4 can be combined or as separate checks. **If combined, must logically work**. BP3 and 4 can be checks for positive or negative. Check must refer to variable for **both** comparisons.
					
					BP5 given if **invalid** output (once or multiple times) for all invalid PINs even if valid also output. BP6 only given if **valid** is output as the only output.
					
					If sensible attempt at BP3 and BP4 (but incorrect), FT to BP5 and 6.
					
					Section A so candidates can respond in pseudocode, structured English, flowcharts, etc. However, must not simply be a repeat of the question.
					
					<u>Examiner’s Comments</u>
					
					As a question in Section A, candidates are free to use a high-level programming language, flowcharts or structured English. Pleasingly, the vast majority attempted to use a high-level programming language and this proved to be very successful for many.
					
					Candidates proved able to take input as a string, to use in-built length functions to check for length and to output messages based on these checks.
					
					The major misconception that affected many students related to checking two invalid PINs, 1234 or 4321. Where two checks are done within a single IF statement it is important that each

Question			Answer/Indicative content	Marks	Guidance
					condition **must** be a valid comparison and therefore include the variable to check against for both comparisons. **This is the same misconception highlighted on Question 1 (b) (i).** **Misconception** Selection statements using two conditions joined together with a Boolean condition such as OR must make sure that both conditions are logically valid comparisons. In many cases, this will mean referencing a variable twice, once in each condition. For example, **`if PIN == "1234" OR "4321"`** is incorrect because the second condition is not a valid comparison – what is being compared to "4321" in order for the condition to be evaluated as True? The correct statement would be **`if PIN == "1234" OR PIN == "4321"`**.
			Total	6	

Question			Answer/Indicative content	Marks	Guidance

2			• input **and store/use** number of tickets needed
			• Check if enough `tickets` left…
			• …sensible confirmation message output correctly for **both** booked and no tickets available
			• …`tickets` variable updated correctly
			• Attempt at iteration…
			• …repeat **all above that has been attempted** <u>while tickets > 0</u> // <u>while tickets != 0</u> // <u>until tickets <=0</u> // <u>until tickets = 0</u>
				6	Allow any suitable confirmation message for BP3 (booked / not enough) but must be right way around.
					
					BP2 must allow tickets to be booked when exact number remain (e.g. 5 tickets needed and 5 remaining). Check comparison operator does not exclude this.
					
					BP3 and 4 dependent on BP2. If BP2 incorrect / not given but reasonable attempt (e.g. checking if <u>`tickets`</u> bigger than 0) allow FT for BP3/4.
					
					Any attempt at loop means giving BP5
					
					BP6 must not repeatedly reset tickets to 500. BP6 must include repeated input (if present).
					
					Ignore superfluous code.
					<u>Example code</u>
					`tickets = 500`
					`while tickets > 0`
					`  num = input("enter num tickets")`
					`  if tickets >= num then`
					`  print("tickets booked")`
					`  tickets = tickets – num`
					`  else`
					`  print("not enough tickets")`
					` endif`
					`endwhile`
					
					<u>Examiner's Comments</u>
					
					This year, candidates appeared much more willing to attempt the question and therefore many received marks for their efforts.
					
					Very few candidates left their response completely blank. This is pleasing to see, as usually we see an increase in the number of candidates who do not attempt this question at all.
					
					Marks were allocated on the mark scheme for the correct input from the user and any attempt at iteration, with most candidates achieving at least the first of these.
					
					Many candidates made a very strong attempt at checking for tickets available.

Question	Answer/Indicative content	Marks	Guidance

			Where this was done correctly, it included the scenario where exactly the available number of tickets were requested, (e.g. 7 tickets remaining and 7 tickets were requested). If, however, a candidate checked **`if tickets > num`** (where num is the number requested), this was not correct as it would not allow this exact scenario.
			
			Other common mistakes included:
			
			• checking if tickets was less than 500 instead of checking against the number requested
			• subtracting the number of tickets required even when the booking was not successful.
			
			Altogether however, this was an well answered question with most candidates achieving some marks and many achieving all or almost all available marks.
			
			Exemplar 3
			
			`tickets = 500`
			`while tickets != 0:`
			`  TNum = int(input('how many tickets'))`
			`  if TNum <= tickets:`
			`  print('tickets booked')`
			`  tickets = tickets - TNum`
			`  else:`
			`  print('Not enough tickets')`
			`endwhile`
			
			In this example, the candidate successfully achieves all mark points for a 6 mark response.
			
			• The number of tickets requested is entered and stored in the variable **`TNum`**.
			• This value is then checked against the **`tickets`** variable, using <= to make sure that there are enough tickets remaining.
			• If this check was successful, a message is printed and the value in tickets reduced correctly.
			• If the check was not successful then a “not enough tickets” message is printed instead.
			• All of this is repeated while the number of **`tickets`** does not equal 0.

Question			Answer/Indicative content	Marks	Guidance
					This was a typical response that achieved full marks.
			Total	6	

Question			Answer/Indicative content	Marks	Guidance
3	a	i	• String • Integer • Real / Float	3 (AO3)	Accept alternative equivalent correct data types (e.g. single/double/decimal for BP3) Do not accept char for BP1 <u>Examiner's Comments</u> This question was completed very well by the vast majority of candidates, showing that the use of data types are now well understood by centres.
		ii	• `theTeam.length() - 1 // 5` • `count` • `studentName` • `True`	4 (AO3)	Accept 6 // `theTeam.length()` for BP1 (Python). Accept alternative length functions e.g. len() Accept **`count = 5`** (and equivalents) for BP1. Accept "True" for BP4. Do not allow obvious spaces in variable names. Ignore capitalisation. <u>Examiner's Comments</u> This was a challenging question revolving around the use of an array that is iterated through to implement a linear search. Many candidates achieved two marks for the first and last points, understanding that the loop would repeat from indexes 0 to 5 and that the value `True` would be returned if the item was found. As usual, allowance was given for off-by-one errors with this loop because of the prevalence of Python in centres and how loops are count controlled loops are handled in this language. It was far less common for candidates to achieve the middle two marks, perhaps because the level of technical knowledge needed was greater. A number of candidates attempted here to fashion a 2D array and refer to multiple indexes, but this was not appropriate given the data structure given. The most challenging mark was certainly the use of `count` as the index of the `theTeam` array, with very few candidates correctly identifying this as the missing element.

Question			Answer/Indicative content	Marks	Guidance
					Assessment for learning Centres are encouraged to link the topics of arrays (both single and two-dimensional) to count controlled loops to be confident in answering questions like this one. A very common programming exercise is to iterate through every item in an array, either to search for an item, count or add items or as the pre-cursor for a search. Candidates are expected to have significant practical programming experience over the duration of their studies.

<table>
<tr><th colspan="3">Question</th><th>Answer/Indicative content</th><th>Marks</th><th>Guidance</th></tr>
<tr><td></td><td>b</td><td></td><td>

- `javelinThrow` **set to 14.3** on **line 01** <u>and</u> `yearGroup` **set to 10 on line 02**
- `score` set to **2** on **line 06**
- `score` set to **4** on **line 11**
- "The score is 4" output on **line 13** with no additional outputs (allow input statements)

<u>Example</u>

<table>
<tr><th>Line number</th><th>javelinThrow</th><th>yearGroup</th><th>score</th><th>Output</th></tr>
<tr><td>01</td><td>14.3</td><td></td><td></td><td></td></tr>
<tr><td>02</td><td></td><td>10</td><td></td><td></td></tr>
<tr><td>06</td><td></td><td></td><td>2</td><td></td></tr>
<tr><td>11</td><td></td><td></td><td>4</td><td></td></tr>
<tr><td>13</td><td></td><td></td><td></td><td>The score is 4</td></tr>
</table>

Answer may include lines where no changes or output happens (i.e. lines 3, 4, 5, 7, 8, 9, 10, 12).

Where variable doesn't change, current value may be repeated on subsequent lines.

</td><td>4
(AO3)</td><td>

Max 3 if in wrong order or additional (incorrect) changes. Penalise line numbers once then FT.

Allow FT for BP4 for current value of `score`.

BP4 must <u>not</u> include comma. Ignore superfluous spaces. Ignore quotation marks.

Treat any entry in output column as an output, even if "x", "-" or "0".

<u>Examiner's Comments</u>

Trace tables have appeared multiple times in J277 examination papers now and candidates are hopefully familiar with the expectations, which are consistent across series.

Most candidates correctly traced through the given algorithm, which was more accessible than usual due to not containing any loops. Where mistakes were made, this was typically to do with either incorrect line numbers being given for each change (this was penalised once only and then subsequent mistakes with line numbers followed through) or additional incorrect output being given.

Candidates should be encouraged to simply leave boxes blank if no output is given on a particular line. If (for example) 'x' is written, examiners are unsure whether the candidate meant no output or the letter 'x' should be output. This ambiguity would mean that no mark could be given.

</td></tr>
<tr><td></td><td>c</td><td>i</td><td>

- inputs a value from the user <u>and stores/uses</u>
- checks min value (>= 40.0 // < 40)
- checks max value (<=180.0 // > 180)
- …outputs **both** valid / not valid correctly <u>based on checks</u>

<u>Example 1 (checking for valid input)</u>

```
h = input("Enter height
jumped")
if h >= 40 and h <= 180 then
  print("valid")
else
```

</td><td>4
(AO3)</td><td>

Answers using `AND/OR` for BP2 and BP3 must be logically correct e.g. `if height >=40 and height <=180`. Do not accept `if height >=40 and <=180`

Answers using OR will reverse output for BP4 (see examples).

BP4 needs reasonable attempt at **either** BP2 or BP3. Need to be sure what is being checked to be able to decide which way around valid/invalid should be.

</td></tr>
</table>

Question			Answer/Indicative content	Marks	Guidance

			`  print("not valid")`
			`endif`
			
			<u>Example 2 (checking for invalid input)</u>
			`h = input("Enter height jumped")`
			`if h < 40 or h > 180 then`
			`  print("not valid")`
			`else`
			`  print("valid")`
			`endif`
					Allow FT for BP4 if reasonable attempt at validating (must include at least one boundary)
					
					Ignore conversion to int on input. `input` cannot be used as a variable name.
					
					Greater than / less than symbols must be appropriate for a high-level language / ERL. Do not accept => (wrong way around) or ≥ (not available on keyboard). No obvious spaces in variable names. Penalise once and then FT.
					
					<u>Examiner's Comments</u>
					
					This question was done very well by the majority of candidates, with multiple ways of achieving the marks possible.
					
					One common problem was consistent with Question 3 (b), as again multiple conditions could be evaluated using a single `if` statement. Candidates needed to refer to their input value for each comparison if they were to achieve full marks. Another common problem was with the boundaries used; the tests must check 40 to 80 inclusive (for valid response) to be marked as correct. Obviously, if an input on these boundaries would produce the wrong output then full marks could not be given.
					
					One final common problem involved the use of greater than or equal to signs (and also less than or equal to). The common signs used in mathematics are not available on a typical keyboard and so would not be allowed in Section B of this paper. Instead, >= or <= should be used, and these should be the correct way around.

		ii	• Any normal value (between 40 and 180 inclusive) • 40.0 // 180.0 • Any value less then 40 // any value greater that 180 // any non-numeric value	3 (AO3)	No need to include decimals, e.g. accept 50. Ignore cm if given. Answer must be actual data (e.g. 50) and not description of data (e.g. "a value between 40 and 180"). If descriptions given, do not accept this as non-numeric for BP3

Question			Answer/Indicative content	Marks	Guidance
	d		• `TeamName` only in first space • `TblResult` in second space • `WHERE` • …`YearGroup` = `11`	4 (AO3)	Max 3 if not in correct order / includes other logical errors. Ignore capitals. Do not accept * or additional fields for BP1 Spelling must be accurate (e.g. not TblResult<u>s</u>). No spaces in field names, penalise obvious spaces once and then FT. Allow quotation marks around field names, table name and 11 Accept == for BP4 (invalid SQL but works in some environments) <u>Examiner's Comments</u> A number of candidates struggled with this question. Problems included spaces in field names, misspelling of the table name (such as `TblResult`<u>s</u> plural when `TblResult` singular was given) or misunderstanding of the WHERE clause. Allowance was given where == was used for comparison and examiners were instructed to allow this (as this is used for comparison in high-level languages such as Python), although this is incorrect as defined in the most recent ANSI SQL standards.

Question			Answer/Indicative content	Marks	Guidance
	e	i	• any **example** of simplification / removing data or focussing on data (in the design) Examples : - “focus on student names and events” - “ignore data such as students’ favourite subjects” - “store year groups instead of ages or DOB” - “shows student IDs instead of full student details”	1 (AO3)	Must be applicable to this program (in the context of students and a sports day), **not** a generic description of what abstraction is. Give BOD where this is unclear. Examiner’s Comments Both questions here asked about abstraction and decomposition of the sports day program. As explained previously in this report, where a scenario or context is given, candidates are expected to use this context. No marks were given by examiners for generic definitions of what the term abstraction or decomposition means. Abstraction in the sports day program could have been for focusing on anything sensible (such as event names) or removing/ignoring anything sensible (such as showing student IDs instead of names). Where the context of the sports day was used, candidates were generally successful in achieving this mark. Decomposition use was more tricky to correctly identify, as many candidates simply referred to how already separate data was stored. Where this extended to true decomposition (such as breaking down data into multiple tables, splitting up event data, etc.) this was credited but the average candidate fell short here. A much more successful approach was to discuss the decomposition of the program, such as having a separate algorithm for each event. Candidates attempting this angle of response did very well. Examiners were instructed for both questions to be generous in deciding whether candidates had indeed referred to the sports day context.
		ii	• any **example** of breaking down the **program** into sections/subroutines • any **example** of breaking down the **database** into tables Examples : - “splits the program up into different events” - “separates the validation routines into subroutines”	1 (AO3)	Must be applicable to this program, not a generic description of what decomposition is. Give BOD where this is unclear. Do not give answers discussing splitting into **fields** (e.g. split into StudentID, YearGroup, etc). BOD if answer discusses one table but suggests other tables could be used.

Question			Answer/Indicative content	Marks	Guidance
			- “breaks the database down into a table per event”		Do not give answers relating simply to data being split into smaller groups unless this clearly relates to how data is decomposed into **tables** in the DB. Allow reference to sports day to mean sports day program. <u>Examiner’s Comments</u> Both questions here asked about abstraction and decomposition of the sports day program. As explained previously in this report, where a scenario or context is given, candidates are expected to use this context. No marks were given by examiners for generic definitions of what the term abstraction or decomposition means. Abstraction in the sports day program could have been for focusing on anything sensible (such as event names) or removing/ignoring anything sensible (such as showing student IDs instead of names). Where the context of the sports day was used, candidates were generally successful in achieving this mark. Decomposition use was more tricky to correctly identify, as many candidates simply referred to how already separate data was stored. Where this extended to true decomposition (such as breaking down data into multiple tables, splitting up event data, etc.) this was credited but the average candidate fell short here. A much more successful approach was to discuss the decomposition of the program, such as having a separate algorithm for each event. Candidates attempting this angle of response did very well. Examiners were instructed for both questions to be generous in deciding whether candidates had indeed referred to the sports day context.

Question			Answer/Indicative content	Marks	Guidance

	f		• Input team name AND score **and** store / use separately
			• Attempt at using iteration…
			• …to enter team/score until "stop" entered
			• Calculates highest score
			• Calculates winning team name…
			• …Outputs highest score **and** team name
			
			<u>Example 1</u>
			`highscore = 0`
			`while team != "stop"`
			`  team = input("enter team name")`
			`  score = input("enter score")`
			`  if score > highscore then`
			`  highscore = score`
			`  highteam = team`
			`  endif`
			`endwhile`
			`print(highscore)`
			`print(highteam)`
			
			<u>Example 2 (alternative)</u>
			`scores = []`
			`teams = []`
			`while team != "stop"`
			`  team = input("enter team name")`
			`  score = input("enter score")`
			`  scores.append(score)`
			`  teams.append(team)`
			`endwhile`
			`highscore = max[scores]`
			`highteam = teams[scores.index(highscore)]`
			`print(highscore)`
			`print(highteam)`
				6
				(AO3)
					For BP3, allow "stop" to be entered for either team name or score (or both). Allow third input (e.g. "do you wish to stop?")
					
					Allow use of `break` (or equivalent) to exit loop for BP3.
					
					Allow use of recursive function(s) for BP2/3.
					
					Initialisation of variables not needed - assume variables are 0 or empty string if not set.
					
					Ignore that multiple teams could get the same high score, assume only one team has the highest score.
					
					BP4/5 could be done in many ways – see examples. Allow any logically valid method. Allow use of max/sum functions and use of arrays/lists.
					
					FT for BP6 if attempt made at calculating highest score/name
					
					If answer simply asks for multiple entries (not using iteration), BP2 and 3 cannot be accessed but all others available.
					
					For minor syntax errors (e.g. missing quotation marks or == for assignment, spaces in variable names) penalise once then FT.
					
					`input` cannot be used as a variable name.
					
					<u>Examiner's Comments</u>
					
					The final question in Section B is expected to be challenging and this proved to be the case, although again was perhaps more accessible than previous papers' final questions.
					
					Marks were available for inputs (one mark) and correctly iterating over as required (two marks), with these three marks proving to be the easiest to achieve. The next three marks required significant processing in terms of calculating the highest score and team name from multiple values entered by the user. The vast majority of candidates simply kept a

Question	Answer/Indicative content	Marks	Guidance

			running ‘highest score’ and updated this on each iteration. Where this was attempted, it was mostly successful. Other candidates attempted more complex solutions, including adding data to arrays and then calculating highest values; where this was done successfully, this of course achieved full marks but frequently small logic mistakes meant that not all marks were given. Centres should encourage candidates to keep their responses simple and not produce over-elaborate solutions if a simpler alternative is available.
			
			A common mistake was where candidates attempted to use loops in the style of `for x in list :`. In this case, the variable x is a reference to an item in the array and not an index. It would therefore not be appropriate to try to access `list[x]` later in the code.
			
			A significant number of candidates were able to create a solution that fully met the requirements of the question, doing so in an elegant and efficient manner. This is extremely pleasing and show excellent understanding, produced from excellent teaching and significant amounts of practical programming experience.
			
			Exemplar 3
			
			9F
			highscore = 0
			highteam = "null"
			while True:
			name = input("please enter a team name or stop to finish: ")
			if name == "stop":
			break
			score = input("please enter with teams score or stop to finish:")
			if score == "stop":
			break
			if int(score) > highscore:
			highscore = int(score)
			highteam = name
			print(highteam + " won with a score of " + str(highscore))
			
			The candidate response shown here achieved six out of six marks. Both the `name` and `score` are input as required, with this being inside a while loop. Perhaps unconventionally (but acceptably), the candidate has used a `break` command to end the loop (which otherwise is infinite) upon stop being entered. This could have been more

Question			Answer/Indicative content	Marks	Guidance
					elegantly rewritten as `while name != 'stop'` but this would not have achieved any further marks. Within the loop, the candidate uses two variables to keep track of the current highest score and associated team, before printing these out in a message once the loop has ended.
			Total	30	

Question			Answer/Indicative content	Marks	Guidance

4			1 mark per bullet point :
			
			• storing both strings correctly in `word1` and `word2`
			• correct concatenation (`word1` **`then`** `word2`)…
			• …storing in variable `message`
			
			<u>Example</u>
			`word1 = "Hello"`
			`word2 = "Everyone"`
			`message = word1 + word2`
				3
				(AO3)
					Accept & / + / . etc as valid methods of concatenation. Allow use of sensible concatenation functions e.g. concat() . **Do not allow commas.**
					
					Do not allow == for assigning value to string. Do not allow spaces in variable names. Penalise once then FT.
					
					Ignore additional code given. Ignore case.
					
					Reasonable attempt at BP2 needed to access BP3.
					
					<u>Examiner's Comments</u>
					
					This question was intended to be straightforward and aimed at all candidates, including those expecting to achieve lower grades. However, a significant number of candidates dropped marks, either through perhaps not reading the question requirements or through not understanding the term concatenation.
					
					There is no requirement in the question to print out the message obtained, simply to store it in the variable `message`. Where this was done in addition, this was not penalised but often it was done instead of storing the concatenated message.
					
					A common mistake was to use the comma for concatenation when in most high-level languages (and especially Python), this is simply used to print out two items or pass two arguments to a subroutine and not actually concatenate values. Another common mistake was with the names of the variables given – if these included spaces or differed from those given in the question (such as `word 1, wordone` or `world1` instead of `word1`) examiners were instructed to not allow these alternatives.

			Total	3	